Cahier des charges

Présentation du projet :

Le contexte : Le client est un responsable des véhicules d'urgences. Il s'agit d'un marché public car la demande du client vient du secteur public par son poste de responsable des véhicules d''urgences telle que les camions de pompiers ou les ambulances ou les véhicules de police...

La problématique : Le besoin du client est de faire en sorte que les trajets des véhicules d'urgences soient minimisés en fonction de la circulation sur la route pour permettre aux véhicules d'urgences d'arriver le plus rapidement du point A au point B. Le but à l'avenir c'est que la solution proposée soit implémenter dans tous les carrefours que l'on peut retrouver.

La solution proposée : Il s'agit de fournir une maquette au client avec un outils de simulation qui permettrait que lorsqu'un véhicule d'urgence arrive à un carrefour le feu passe automatiquement au vert en fonction de l'endroit où se trouve le véhicule d'urgence pour fluidifier la circulation et permettre au véhicule d'urgence de passer cet endroit sensible le plus rapidement possible. L'outils de simulation serait déployé dans un carrefour connecté dans une ville de taille moyenne.

Spécifications techniques

- **Langage de base :** Python *(contrainte imposée, spécifiez la version retenue)*.
  Le langage utilisé pour développer l'outils de simulation va être programmé avec le langage Python. La version Python utilisée est 3.14.7.

- **Architecture technique et organisationel d'un point technique (clients, serveurs, ...)**
  L'application repose sur une architecture distribuée client-serveur répartie sur deux VM Linux (Debian) :
  - VM 1 (Serveur & BDD) : héberge le moteur de simulation (calculs physiques et règle de priorité), le serveur Sockets TCP écoutant sur le port 5000 (diffusion de l'état en JSON) ainsi que la base de données MariaDB pour le stockage des statistiques.
  - VM 2 (Client & IHM) : héberge l'interface graphique utilisateur et un client Sockets TCP exécuté dans un thread dédié pour traiter le flux de données en temps réel sans bloquer l'affichage.

  L'ensemble est déployé de manière automatisée sur chaque VM via des scripts Bash d'installation (setup_vm1_server.sh et setup_vm2_client.sh).

- **Framework et librairies envisagées :** choix des outils techniques selon le besoin (ex: Flask/FastAPI pour le web, TkinPyQt pour une interface graphique, Pandas pour la data, etc.).

I**nterface graphique (VM 2) :** PyQt6 pour le rendu visuel dynamique du réseau routier et l'affichage du tableau de bord de statistiques.

Gestion de la base de données (VM 1) : mysql-connector-python pour assurer la connexion et l'exécution des requêtes SQL entre le moteur Python et le serveur MariaDB.

Communication réseau & Concurrence : Modules standards Python socket (transfert TCP des données au format JSON) et threading pour traiter le flux en arrière-plan sans figer l'IHM.

Calculs mathématiques & GPS : Module standard math pour la conversion des coordonnées GPS en pixels.

- **Environnement de développement :** choix de l'IDE, gestion de l'environnement virtuel (venv, conda) et des dépendances (requirements.txt).

IDE : PyCharm pour l'ensemble de l'équipe afin de faciliter le développement local.

Environnement virtuel : Utilisation du module standard venv pour isoler les dépendances Python des paquets système, aussi bien en local sur les postes de travail que sur les VM Linux (Debian).

Gestion des dépendances : Centralisation de l'ensemble des paquets nécessaires (PyQt6, mysql-connector-python, etc.) dans un fichier requirements.txt. Son installation est automatisée sur chaque VM via la commande pip install -r requirements.txt intégrée aux scripts Bash de déploiement.

- **Contraintes techniques annexes :** choix de la base de données (ex: SQLite, PostgreSQL), gestionnaire de version (Git/GitHub), et compatibilité système (Windows, macOS, Linux).

Nous allons utiliser MariaDb pour la base de données. On a créé un projet GitHub pour mettre à jour les modifications du projet de plus on a une branche personnalisée pour chacun de nous pour coder sans gêner les collègues avec une branche principale quand le code est parfait. On a prévu d'utiliser 2 VM Linux pour la communication des sockets mais aussi pour l'installation de l'application avec des scripts Bash pour installer avec une documentation pour ajouter l'installation des scripts Bash.

Périmètre fonctionnel & "Coût humain"

Fonctionnalité	**Priorité (MoSCow)**	**Complexité (1 à 5)**	**Coût humain estimé (Heures)**
Participation à la rédaction du Cahier des charges (Partie technique).	Must Have	3	6 heures
Développement du moteur de simulation en Python (carrefours, densité du trafic).	Must have	5	8 heures

Algorithme de calcul d'itinéraire optimal (Point A vers Point B).	Must have	4	4 heures
Cartographie logicielle / Spécifications de l'IA graphie.	Should have	2	2 heures
Développement des scripts PowerShell pour la gestion de l'autorisation d'accès et création des VM Linux.	Should have	3	4 heures
Conception et interfaçage de la Base de Données (stockage des feux, accès, usagers).	Must have	4	8 heures
Gestion et structuration du dépôt GitHub.	Must have	2	2 heures
Rédaction de la Doc d'installation (docs/installation.md) et de la Doc d'usage (docs/usage.md).	Should have	3	8 heures
Modélisation de la logique V2I (feux tricolores) et V2V (alerte usagers).	Must have	4	6 heures
Rédactions des comptes-rendus hebdomadaire.	Must have	2	10 heures
Elaboration du livrable finances.	Should have	2	2 heures
Rédaction de la Doc de réponse (docs/reponses.md).	Should have	3	6 heures

Planning et livrables

Les dates limites (Deadline) :

Livrable	**Responsable principal**	**Relecture / Validation**	**Echéance**
Cahier des charges.	Lucas Bucher	Quentin Altinok & Benjamin Deiber	23/09/2026
Algorithme itinéraire (simulation.py) & Logique métier avec les classes (models/) & Moteur (main.py).	Lucas Bucher	Quentin Altinok & Benjamin Deiber	15/10/2026
Script PowerShell (setup_vm1_server.sh, setup_vm2_client.sh) & Base de données	Benjamin Deiber	Lucas Bucher & Quentin Altinok	25/10/2026

(database/) et interface graphique (gui/).			
Comptes rendus hebdomadaire CR1.	Quentin Altinok	Benjamin Deiber & Lucas Bucher	13/09/2026
Comptes rendus hebdomadaire CR2.	Quentin Altinok	Benjamin Deiber & Lucas Bucher	21/09/2026
Comptes rendus hebdomadaire CR3.	Quentin Altinok	Benjamin Deiber & Lucas Bucher	4/10/2026
Comptes rendus hebdomadaire CR4.	Quentin Altinok	Benjamin Deiber & Lucas Bucher	30/10/2026
Comptes - Rendus hebdomadaire CR5.	Quentin Altinok	Benjamin Deiber & Lucas Bucher	15/11/2026
Livrable Finance.	Quentin Altinok	Lucas Bucher & Benjamin	15/11/2026
Documentations (README, Docs).	Benjamin Deiber	Lucas Bucher & Quentin Altinok	15/11/2026
Dépôt GitHub partagé (fdrouhin-uha) .	Benjamin Deiber	Lucas Bucher & Quentin Altinok	15/11/2026
Réseau (network/) & Mathématiques GPS (utils/).	Quentin Altinok	Lucas Bucher & Benjamin Deiber	25/10/2026
IA graphie / Cartographie logicielle.	Lucas Bucher	Benjamin Deiber & Quentin Altinok	25/10/2026

La Timeline (Rétroplanning) :

Mission	Cahier des charges.	Algorithme itinéraire (simulation.py) & Logique métier avec les classes (models/) & Moteur (main.py).	Script PowerShell (setup_vm1_server.sh, setup_vm2_client.sh) & Base de données (database/) et interface graphique (gui/).	Comptes - Rendus Hebdomadaire .	Réseau (network/) & Mathématiques GPS (utils/).	Documentations (README, Docs).	IA graphie / Cartographie logicielle.
Semaines							
07/09/2026 Jusqu'au 13/09/2026	X		X	X	X		
14/09/2026 Jusqu'au 20/09/2026	X	X	X	X	X		
21/09/2026 Jusqu'au 27/09/2026	X	X	X	X	X		
28/09/2026 Jusqu'au 04/10/2026		X	X	X	X		
05/10/2026 Jusqu'au 11/10/2026		X	X		X		
12/10/2026 Jusqu'au 18/10/2026		X	X		X		X
19/10/2026 Jusqu'au 25/10/2026			X		X	X	X
26/10/2026 Jusqu'au 1/11/2026				X		X	
2/11/2026 Jusqu'au 8/11/2026						X	

9/11/2026 Jusqu'au 15/11/2026				X		X	